



CRÉATION D'UNE API REST AVEC LARAVEL

Rapport de Développement

Présenté par :
Yahya El Omari

Encadré par :
Abdelmajid Bendrif

23 février 2026

Table des matières

1	Introduction et Vue d'Ensemble	2
2	Étape 1 : Conception de l'Architecture	2
3	Étape 2 : Couche Base de Données	3
3.1	1. Les Migrations	4
3.2	2. Modèles et Relations Eloquent	4
3.3	3. Factories et Seeders	4
4	Étape 3 : Requêtes GET Optimisées	5
4.1	1. Ressources JSON et Collections	5
4.2	2. Création de l'ApiFilter	6
4.3	3. Eager Loading Conditionnel	7
5	Étape 4 : Gestion des Requêtes CRUD (Écriture)	8
5.1	1. Requêtes POST	8
5.2	2. Requêtes UPDATE (PUT) et PATCH	8
5.3	3. Requêtes DELETE	8
6	Conclusion Générale	9

1 Introduction et Vue d'Ensemble

Ce guide technique retrace l'implémentation complète d'une API REST de démonstration en utilisant le framework Laravel. La documentation décrit étape par étape la création de l'architecture, de la base de données jusqu'à l'exposition des endpoints avec système de filtrage complexe, pagination personnalisée et gestion rigoureuse du cycle CRUD. Les exemples de code se concentrent sur l'entité **Vehicle**, mais la même logique a été dupliquée pour l'ensemble des autres modèles (*Category*, *Location*, *Review* et *VehicleDetail*).

2 Étape 1 : Conception de l'Architecture

Avant de coder, une analyse des besoins a permis d'établir les relations entre les entités.

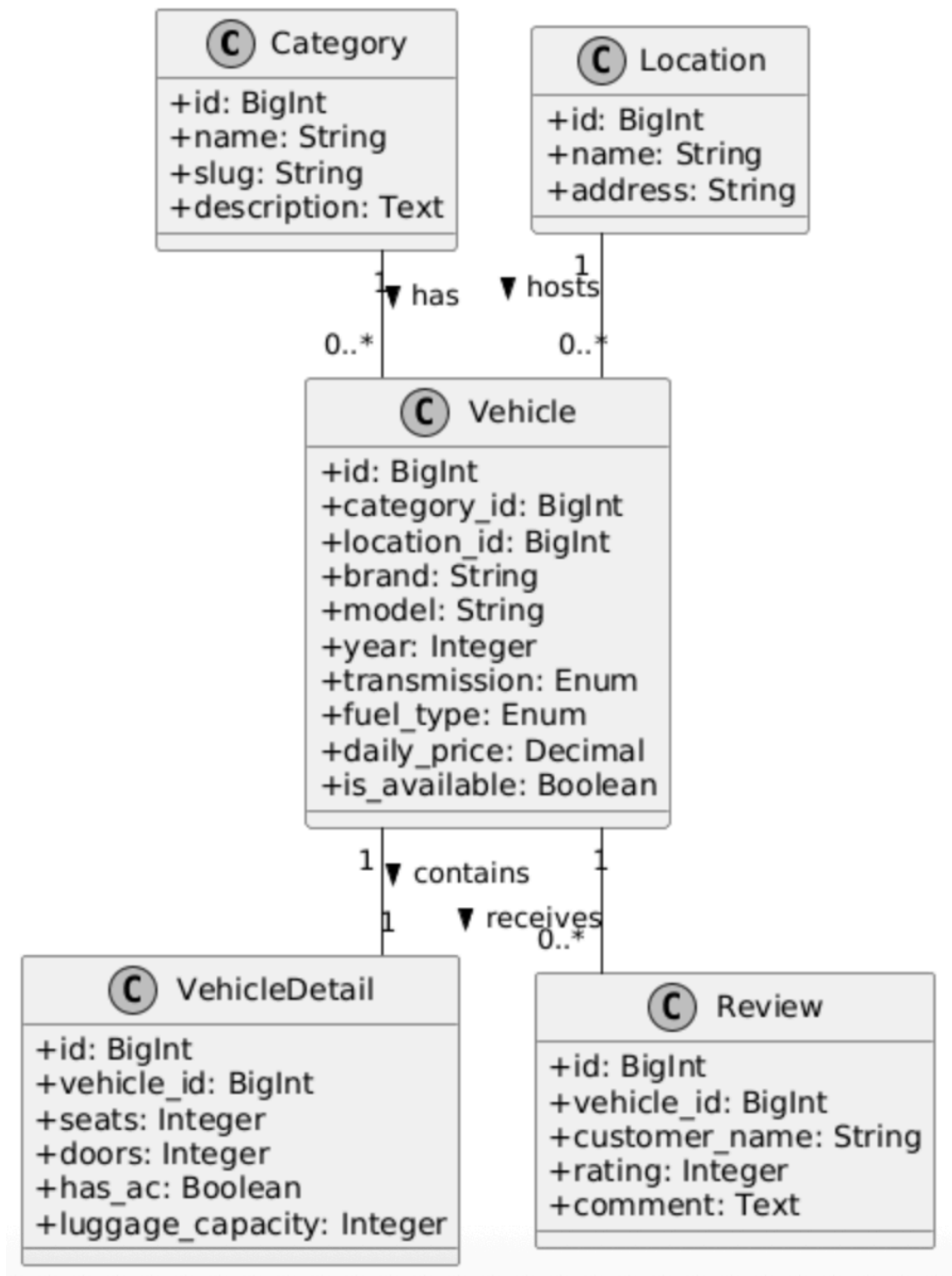


FIGURE 1 – Diagramme de Classes du Système

Ce diagramme met en évidence la centralité de la table 'vehicles' et ses liens avec les emplacements, les catégories et les détails.

3 Étape 2 : Couche Base de Données

3.1 1. Les Migrations

La première étape concrète a consisté à créer la structure des tables. Les migrations assurent la présence des clés étrangères.

```
1 public function up() {
2     Schema::create('vehicles', function (Blueprint $table) {
3         $table->id();
4         $table->foreignId('location_id')->constrained();
5         $table->foreignId('category_id')->constrained();
6         $table->string('brand');
7         $table->string('model');
8         $table->integer('year');
9         $table->string('transmission');
10        $table->decimal('daily_price', 8, 2);
11        $table->boolean('is_available')->default(true);
12        $table->timestamps();
13    });
14 }
```

Listing 1 – Migration create_vehicles_table.php

3.2 2. Modèles et Relations Eloquent

Ensuite, nous avons configuré les modèles avec les propriétés `$fillable` (pour autoriser le Mass Assignment lors des POST/PUT) et défini les relations Eloquent.

```
1 class Vehicle extends Model {
2     protected $fillable = [
3         'location_id', 'category_id', 'brand', 'model',
4         'year', 'transmission', 'daily_price', 'is_available'
5     ];
6
7     public function category() { return $this->belongsTo(Category::class); }
8     public function location() { return $this->belongsTo(Location::class); }
9     public function vehicleDetail() { return $this->hasOne(VehicleDetail::class); }
10    public function reviews() { return $this->hasMany(Review::class); }
11 }
```

Listing 2 – Le Modèle Vehicle.php

3.3 3. Factories et Seeders

Pour alimenter la base de données avec des jeux d'essais volumineux, des *Factories* utilisant le package 'Faker' ont été générées.

```
1 class VehicleFactory extends Factory {
2     public function definition(): array {
3         return [
4             'location_id' => Location::factory(),
5             'category_id' => Category::factory(),
6             'brand' => $this->faker->word(),
7             'model' => $this->faker->word(),
8             'year' => $this->faker->numberBetween(2000, 2024),
9             'transmission' => $this->faker->randomElement(['manual', 'automatic']),
10            'daily_price' => $this->faker->numberBetween(300, 1000),
11            'is_available' => $this->faker->boolean(),
12        ];
13    }
14 }
```

Listing 3 – VehicleFactory.php (Génération Faker)

Le `DatabaseSeeder` orchestre ensuite l'exécution de ces factories dans le bon ordre pour respecter les contraintes de clés étrangères.

```
1 class DatabaseSeeder extends Seeder {  
2     public function run(): void {  
3         $this->call([  
4             VehicleSeeder::class,  
5             CategorySeeder::class,  
6             LocationSeeder::class,  
7             // ... autres seeders  
8         ]);  
9     }  
10 }
```

Listing 4 – DatabaseSeeder.php

4 Étape 3 : Requêtes GET Optimisées

4.1 1. Ressources JSON et Collections

Pour standardiser les réponses de l'API et masquer la structure interne de la base de données (ex : transformer le format `snake_case` en `camelCase` pour le frontend), nous avons utilisé les `JsonResource` de Laravel.

Pour une entité unique, le `VehicleResource` gère le mapping de chaque attribut pertinent. Il inclut également la logique pour charger les relations conditionnellement via `whenLoaded()`, empêchant ainsi le déclenchement de requêtes non souhaitées si la relation n'a pas été explicitement demandée.

```
1 class VehicleResource extends JsonResource {  
2     public function toArray(Request $request): array {  
3         return [  
4             'id' => $this->id,  
5             'locationId' => $this->location_id,  
6             'brand' => $this->brand,  
7             'dailyPrice' => $this->daily_price,  
8             'vehicleDetail' => new VehicleDetailResource($this->whenLoaded('vehicleDetail')),  
9             'reviews' => ReviewResource::collection($this->whenLoaded('reviews'))  
10        ];  
11    }  
12 }
```

Listing 5 – Mapping dans VehicleResource.php

Pour le renvoi de listes d'éléments, nous utilisons une `ResourceCollection`. Couplée avec la méthode `paginate()`, elle enveloppe automatiquement la liste des véhicules de données meta utiles pour la navigation de la pagination côté client, tout limitant la charge serveur.

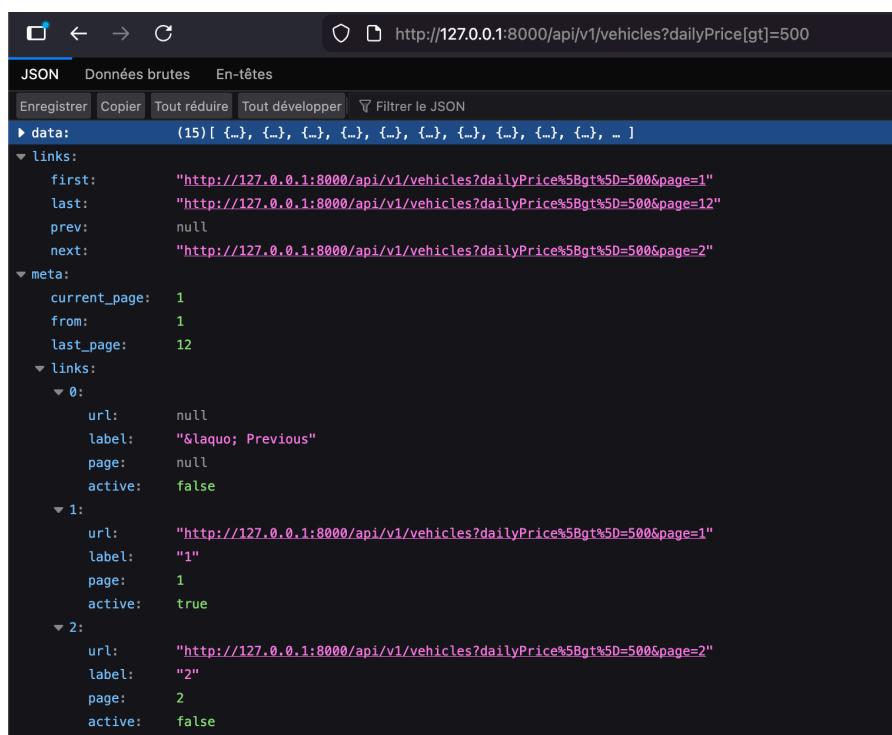


FIGURE 2 – Exemple de Collection Paginée dans Postman

4.2 2. Création de l'ApiFilter

Nous avons implémenté une logique permettant aux clients de filtrer dynamiquement les résultats via l'URL (par exemple : `?year[gt]=2020`). Le cœur de ce système réside dans la classe parent `ApiFilter` qui transforme les paramètres de la requête web en conditions Eloquent ('where').

```

1 class ApiFilter {
2     public function transform($request) {
3         $eloquentQuery = [];
4         foreach ($this->safeParms as $parm => $operators) {
5             $query = $request->query($parm);
6             if (!isset($query)) continue;
7
8             $column = $this->columnMap[$parm] ?? $parm;
9
10            if (!is_array($query)) {
11                $eloquentQuery[] = [$column, $this->operatorMap['eq'], $query];
12                continue;
13            }
14            // ... Boucle secondaire pour traiter les opérateurs (gt, lt, etc.)
15        }
16        return $eloquentQuery;
17    }
18 }

```

Listing 6 – Logique de transformation (ApiFilter.php)

Chaque modèle possède ensuite sa propre sous-classe (ex : `VehicleFilter`) pour définir intelligemment quels champs et opérateurs sont autorisés, et assurer le mapping camelCase.

```

1 class VehicleFilter extends ApiFilter {
2     protected $safeParms = [
3         'locationId' => ['eq'],
4         'year' => ['eq', 'gt', 'lt', 'gte', 'lte'],
5         // ...

```

```

6     ];
7
8     // Mappe les noms de l'API vers la BDD
9     protected $columnMap = [
10         'locationId' => 'location_id',
11     ];
12 }

```

Listing 7 – La sous-classe VehicleFilter.php

4.3 3. Eager Loading Conditionnel

Pour résoudre le problème "N+1", le chargement des relations est devenu conditionnel. Le Request injecte directement les relations (ex : `includeReviews=true`).

```

1 public function index(Request $request) {
2     $filter = new VehicleFilter();
3     $queryItems = $filter->transform($request);
4
5     $vehicle = Vehicle::where($queryItems);
6
7     if ($request->query('includeVehicleDetails')) {
8         $vehicle = $vehicle->with('vehicleDetail');
9     }
10
11     return new VehicleCollection($vehicle->paginate()->withQueryString());
12 }

```

Listing 8 – Contrôleur gérant l'Eager Loading

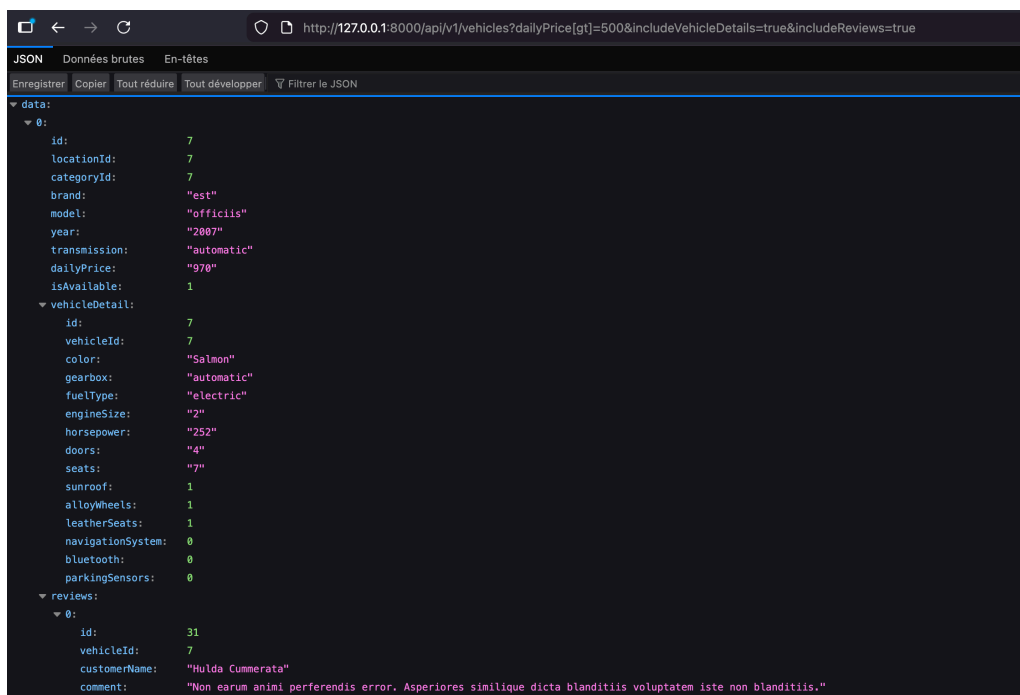


FIGURE 3 – Chargement Conditionnel de vehicleDetail

5 Étape 4: Gestion des Requêtes CRUD (Écriture)

5.1 1. Requêtes POST

Pour l'ajout de ressources, nous utilisons des `FormRequest`. La méthode `prepareForValidation()` traduit les paramètres camelCase issus du JSON en paramètres snake_case compatibles avec la base de données.

```
1 class StoreVehicleRequest extends FormRequest {  
2     public function rules(): array {  
3         return [  
4             'location_id' => 'required|exists:locations,id',  
5             'daily_price' => 'required|numeric',  
6             // ...  
7         ];  
8     }  
9  
10    protected function prepareForValidation() {  
11        $this->merge([  
12            'location_id' => $this->locationId,  
13            'daily_price' => $this->dailyPrice,  
14        ]);  
15    }  
16 }
```

Listing 9 – StoreVehicleRequest.php

5.2 2. Requêtes UPDATE (PUT) et PATCH

La logique de modification permet de gérer les mises à jour intégrales (`PUT`) ou partielles (`PATCH`) par un système de conditions dans les règles de validation.

```
1 public function rules(): array {  
2     if ($this->method() == 'PUT') {  
3         return [  
4             'brand' => 'required|string|max:255',  
5             'daily_price' => 'required|numeric',  
6         ];  
7     } else {  
8         return [  
9             'brand' => 'sometimes|required|string|max:255',  
10            'daily_price' => 'sometimes|required|numeric',  
11        ];  
12    }  
13 }
```

Listing 10 – UpdateVehicleRequest.php (Logique Conditionnelle)

5.3 3. Requêtes DELETE

La suppression s'effectue via Route Model Binding dans le contrôleur.

```
1 public function destroy(Vehicle $vehicle) {  
2     $vehicle->delete();  
3     return response()->json(null, 204);  
4 }
```

Listing 11 – VehicleController.php (Méthode Destroy)

6 Conclusion Générale

Le strict respect de cette démarche pas-à-pas sur l'ensemble des entités permet d'obtenir un backend extrêmement prédictible. Laravel démontre ici sa capacité à fournir des outils puissants et élégants (FormRequests, JsonResources, Eloquent, Factories) pour la mise en place d'une API REST de qualité professionnelle.